

电子系统导论实验报告

实验 1 【实验名】



指导教师: 胡来归

学生姓名: 徐锦云 学生姓名: 周熙 学生姓名: 敖艺珊

学 号: 学 号: 学 号: _

23307130447 23307130489 23307130507

专 业: 技术科学 专 业: 技术科学 专 业: 技术科学

试验班 试验班 试验班

日 期: 2024.

一、实验目的:

- 1、通过PID控制电机驱动,用摄像头、超声等传感设备使小车识别障碍物;
- 2、根据要求(红色右转,蓝色左转,黄色右转)避开障碍物,冲过终点线。

二、实验原理:

1、硬件部分:

(1) 树莓派:树莓派是一种基于 ARM 处理器的微型计算机,由英国的树莓派基金会开发。它拥有丰富的 GPIO(通用输入输出)引脚,可以用于连接各种外部设备。除了 GPIO,树莓派还具有 USB 端口、HDMI 输出、以太网接口等,使其成为一个功能强大的嵌入式系统。

(2) GPIO 引脚:GPIO 引脚是树莓派上用于连接外部电子设备的通用引脚。通过 GPIO 引脚,可以实现与外部电路的通信和控制,如连接电机、传感器等。

(3) 电机驱动:电机驱动模块将树莓派的输出信号转换为电机可以理解的信号,控制电机的转速和方向。使用 PWM 技术可以调整电机的转速,通过改变脉冲宽度来控制电机的速度和方向。

(4) 摄像头:摄像头模块用于实时获取小车行驶路径的图像数据。摄像头采集到的图像数据将被送入计算机进行图像处理和分析,以实现车辆的自动导航功能。

2、软件部分:

(1) OpenCV 库:OpenCV (Open Source Computer Vision Library) 是一个开源的计算机视觉库,提供了丰富的图像处理和分析功能。OpenCV 用于图像处理和 分析,包括颜色空间转换、图像分割、形态学操作等。

(2) PID 算法:PID (Proportional-Integral-Derivative) 是一种经典的控制算法,常用于控制系统中的闭环控制。PID 算法根据摄像头获取的图像信息,计算车辆当前位置与预期位置之间的偏差,并调整电机的输出信号,使车辆尽量保持在预期路径上行驶。

(3) 图像处理:图像处理部分对摄像头获取的图像数据进行处理,以提取出车辆行驶路径的信息。利用颜色分割和形态学操作等技术,将摄像头采集到的图像数据中的橙色部分提取出来,并进行后续处理,以便后续的路径规划和导航。

3、算法流程:

(1) 初始化:初始化 GPIO 引脚模式和初始参数,准备好摄像头。

(2) 图像处理:获取摄像头图像,检测到符合规则的图形,找到魔方中线,识别颜色。

(3) 计算偏差:根据处理后的图像,计算车辆当前位置与预期位置的偏差。

(4) PID 调节:根据偏差,利用 PID 算法计算出电机的输出调节量,使车辆尽量保持在预期避障的路径上。

(5) 控制电机:根据 PID 调节量,调整电机的输出,控制车辆的速度和方向,使其根据障碍物的颜色,位置来按规则避开。

(6) 循环执行:不断重复以上步骤,实现车辆的自动避障导航功能。

三、实验内容:

小车进行魔方绕桩竞速。场地纵向三等分，第一个从放置在第一条线正中央处红色魔方右侧经过，第二个从第二条线右侧三等分点处蓝色魔方左侧经过，第三个从第三条线右侧三等分点处黄色魔方右侧经过。

1. 实物图:



2. 代码:

代码分为两个, AGV.py 是运行主代码, pid.py 是作为 PID 控制器被 AGV 调用

PID 控制器: pid.py

```
1 class PID:
2     #PID控制器 方便调用与PID控制有关函数
3
4     def __init__(self, P=0.5, I=0, D=0, target=320, init_output=0):
5         # 初始化 PID 控制器的参数
6         self.Kp = P # 比例增益
7         self.Ki = I # 积分增益
8         self.Kd = D # 微分增益
9         self.err_pre = 0 # 上一次误差
10        self.err_last = 0 # 上一次的误差
11        self.u = 0 # 控制器的输出
12        self.integral = 0 # 积分项
13        self.ideal = target # 设定的目标
14        self.last_output = init_output # 上一次的输出
15        self.pre_output = init_output # 当前的输出
16
17
18    def update(self, feedback_value):
19        self.err_pre = self.ideal - feedback_value # 计算当前误差
20        self.integral *= 0.86 # 对积分项进行衰减
21        self.integral += self.err_pre # 更新积分项
22        self.u = self.Kp * self.err_pre + self.Ki * self.integral + \
23            self.Kd * (self.err_pre - self.err_last) # 计算控制器输出
24
25        self.err_last = self.err_pre # 更新上一次误差
26        self.pre_output = self.u # 更新当前输出
27        self.last_output = self.pre_output # 更新上一次输出
28        return self.pre_output # 返回当前输出
29
30    def setKp(self, proportional_gain):
31        #确定PID控制器在设置比例增益时对当前误差作出的响应程度
32        self.Kp = proportional_gain
33
34    def setKi(self, integral_gain):
35        #确定PID控制器在设置积分增益时对当前误差作出的响应程度
36        self.Ki = integral_gain
37
38    def setKd(self, derivative_gain):
39        #确定PID控制器在设置微分增益时对当前误差作出的响应程度
40        self.Kd = derivative_gain
```

运行主代码: AGV.py

```
1 import RPi.GPIO as GPIO # 导入GPIO库
2 import cv2 # 导入OpenCV库
3 import cube_recognizer.libRecognize as lR # 导入魔方识别库
4 import math # 导入数学库
5 from collections import OrderedDict # 导入有序字典
6 from pid import PID # 导入PID控制器
7
8
9 EA, I2, I1, EB, I4, I3 = (13, 19, 26, 16, 20, 21) # 定义引脚
10 FREQUENCY = 50 # PWM频率
11
12 # PID 参数
13 kp = 0.4 # 比例系数
14 ki = 0.001 # 积分系数
15 kd = 0.3 # 微分系数
16 target_x = 320 # 始终不变的量 指向最中央
17
18 #初始阶段速度和转向限制
19 speed_init = 70 # 初始速度
20 lim_init = 18 # 初始转向限制
21
22 stage_cutter = 9 # 第几次循环开始阶段2
23 # 因为小车起步重心不稳为防止抬头分为两个阶段
24
25 # 阶段2速度和转向限制
26 speed_st2 = 70 # 阶段2速度
27 lim_st2 = 27 # 阶段2转向限制
28
29 side_cutter = 70 # 边沿设定
30
31 # 与魔方目标距离系数
32 distant_factor = 2.2 # 与魔方目标距离系数, 越大弯越大
33
34 # 目标坐标限制
35 x_lim_from_side = 67 # 目标坐标限制, 越大越晚转
36
37 # 设置GPIO编号模式
38 GPIO.setmode(GPIO.BCM)
39
40 # 设置GPIO口为输出
41 GPIO.setup([EA, I2, I1, EB, I4, I3], GPIO.OUT)
42 GPIO.output([EA, I2, EB, I3], GPIO.LOW)
43 GPIO.output([I1, I4], GPIO.HIGH)
44
```

```
44 # 设置PWM引脚和频率
45 pwma = GPIO.PWM(EA, FREQUENCY)
46 pwmb = GPIO.PWM(EB, FREQUENCY)
47 pwma.start(0)
48 pwmb.start(0)
49
50 # 打开摄像头
51 cap = cv2.VideoCapture(0)
52
53 ret, frame = cap.read() # 读取摄像头帧
54 ct = 0 # 初始化循环计数器
55
56 cur_x = target_x # 当前X坐标初始化为图像中央
57 # 打开/新建视频文件用于写入,帧率=17,帧尺寸=640x480
58 out = cv2.VideoWriter("movie.avi", cv2.VideoWriter_fourcc(
59     'X', 'V', 'I', 'D'), 17, (640, 480))
60
61 def get_motor_value(adjustment, speed, lim):
62     # 定义函数 get_motor_value
63     if abs(adjustment) > lim: # 如果调整值adjustment的绝对值超过了限制 lim
64         # 将调整值限制在 -lim 到 lim 之间
65         adjustment = lim if adjustment > 0 else -lim
66     # 返回计算后的电机速度,包括增加调整值后的速度和减去调整值后的速度
67     return speed + adjustment, speed - adjustment
68
69 pid_line = PID(kp/100, ki/100, kd/100, target_x, 0) # 初始化PID控制器
70
71 print("Ready")
72 input() # 等待用户输入继续执行程序
73
74 adjust_prev = 0 # 上一次调整量
75 adjust = 0 # 调整量
76 prev_x = cur_x # 上一次X坐标初始化为当前X坐标
77 tgt_x = 320 # 目标X坐标
78 rCN = lR.CubeRecognizer() # 初始化魔方识别器
79 rCN.init() # 初始化魔方识别器
80 count = 0 # 计数器
81
82
83 try:
84     while True: # 进入循环
85         if ct < stage_cutter: # 如果当前循环计数 ct 小于阶段切换值 stage_cutter
86             speed = speed_init # 设置速度为初始速度
87             lim = lim_init # 设置限制为初始转向限制
88         else: # 否则,即当前循环计数 ct 大于等于阶段切换值 stage_cutter
89             speed = speed_st2 # 设置速度为阶段2速度和转向限制
90             lim = lim_st2 # 设置限制为阶段2转向限制
91         ct = ct + 1 # 计数器ct加1
```

```
91
92     ret, frame = cap.read() # 读取摄像头, 返回值存储在 ret 和 frame 中
93     count += 1 # 计数器count加1
94
95     # 调用 get_rec_cen() 获取魔方信息
96     cur_x, color, size, height = rCN.get_rec_cen(frame)
97     # cur_x 是当前魔方中心位置, 通过cur_x=(black_start+black_end)/2计算得出
98     # color 是魔方颜色, size 是魔方的尺寸, height 是魔方的高度
99
100    im = frame # 将当前帧保存为 im
101
102
103    font = cv2.FONT_HERSHEY_SIMPLEX # 设置字体为 OpenCV 内置的简单字体
104
105    # 如果当前识别到魔方且其尺寸在合理范围内
106    if (cur_x != 0 and size < 80000 and size > 1400):
107        # 在图像上绘制垂直于魔方中心的绿色线条
108        cv2.line(im, (cur_x, 0), (cur_x, 480), (0, 255, 0), 3)
109        # 记录当前魔方中心位置为 prev_x, 以便在未检测到魔方时保持转向
110        prev_x = cur_x
111        # 在图像上添加文字显示当前魔方中心的 x 坐标
112        cv2.putText(im, "cur_x:"+str(cur_x),
113                    (10, 30), font, 1, (0, 255, 0), 2)
114
115    # 给tgt_x赋值, 确定转向
116    if cur_x > 640 - side_cutter: # 如果魔方中心靠近图像右侧边缘
117        tgt_x = 320 # 设置目标转向位置为图像中心
118    elif cur_x < side_cutter: # 如果魔方中心靠近图像左侧边缘
119        tgt_x = 320 # 设置目标转向位置为图像中心
120        print("go straight") # 打印信息表示直行
121    # 上列代码为判断方块是否位于图像边缘, 位于边缘则直行
122
123    elif color == "blue": # 如果魔方颜色为蓝色
124        # 根据魔方大小调整目标转向位置向左偏移
125        tgt_x = cur_x - distant_factor * math.sqrt(size)
126    elif color == "red" or color == "yellow": # 如果魔方颜色为红色或黄色
127        # 根据魔方大小调整目标转向位置向右偏移
128        tgt_x = cur_x + distant_factor * math.sqrt(size)
129
130    # 检测tgt_x是否越界
131    # 越界参数可调整(即x_lim_from_side)
132    if tgt_x > 640 - x_lim_from_side:
133        tgt_x = 640 - x_lim_from_side
134    elif tgt_x < x_lim_from_side:
135        tgt_x = x_lim_from_side
136
```

```
136         # 在图像上添加文字
137         cv2.putText(im, "height:"+str(height),
138                     (10, 120), font, 1, (0, 0, 255), 2) #显示魔方高度
139         cv2.putText(im, "size:{:.1f}".format(size),
140                     (10, 150), font, 1, (0, 0, 255), 2) #显示魔方尺寸
141
142         adjust = pid_line.update(tgt_x) # 使用 PID 控制器更新转向调整量
143         adjust_prev = adjust # 记录当前的转向调整量作为上一个调整量
144
145     elif size >= 30000: # 如果检测到的魔方尺寸过大(异常情况)
146         adjust = 0 # 不进行转向调整
147     else: # 如果当前未识别到魔方
148         if count >= 20: # 如果计数器大于等于 20
149             tgt_x = 320 # 设置目标转向位置为图像中心
150             speed = speed_init # 恢复默认速度
151             lim = lim_init # 恢复默认限制
152             # 根据之前记忆的魔方位置进行转向调整
153             adjust = pid_line.update(0.9 * prev_x)
154             if prev_x < 320: # 如果之前的魔方位置在图像左侧
155                 tgt_x = 80 # 设置目标转向位置为较左侧位置
156             else: # 如果之前的魔方位置在图像右侧
157                 tgt_x = 535 # 设置目标转向位置为较右侧位置
158
159         # 根据目标转向位置进行转向调整
160         adjust = pid_line.update(1.12 * tgt_x)
161
162         # 在图像im中tgt_x位置上画一条垂直线
163         cv2.line(im, (int(tgt_x), 0), (int(tgt_x), 480), (255, 0, 0), 3)
164
165         # 添加文字, 标注tgt_x值
166         cv2.putText(im, "tgt_x:"+str(tgt_x), (10, 60),
167                     font, 1, (255, 0, 0), 2)
168         cv2.putText(im, "adj:{:.3f}".format(adjust),
169                     (10, 90), font, 1, (0, 0, 255), 2)
170
171         # 根据adjust乘以lim的结果和指定的速度计算两个电机的速度值
172         motor_a, motor_b = get_motor_value(adjust*lim, speed, lim)
173
174         # 设置PWM信号, 改变电机的占空比
175         pwma.ChangeDutyCycle(motor_a) #电机a占空比为motor_a
176         pwmb.ChangeDutyCycle(motor_b) #电机b占空比为motor_b
177
178         # 将带有注释和文字的图像im写入视频输出
179         out.write(im)
180 except KeyboardInterrupt:
181     print("End")
182
183 #释放清理
184 cap.release()
185 out.release() # 关闭视频文件
186 cv2.destroyAllWindows()
187 pwma.stop()
188 pwmb.stop()
189 GPIO.cleanup()
190 rCN.terminate()
```

四、 设计思路:

本实验基于树莓派 (Raspberry Pi) 和 Python 语言, 结合了硬件控制 (使用 RPi.GPIO 库) 和图像处理 (使用 OpenCV 库)。目标是通过图像识别 (摄像头识别魔方颜色) 实现小车的自动避障绕桩驾驶, 并采用 PID 控制算法来调整小车的运动轨迹

1、硬件设置

①树莓派 GPIO 引脚用于连接小车的电机控制器, 包括左右两个电机的控制。

②采用 PWM (脉冲宽度调制) 控制电机速度, 以实现精确的加速和减速。

2、图像处理与魔方识别

小车搭载摄像头, 通过 OpenCV 库捕获实时图像, 并进行色彩空间转换 (从 BGR 到 HSV), 以便于后续魔方识别库对图像处理得到魔方相关参数 (中心位置、颜色、尺寸、高度)。

3、避障逻辑

①如果检测到符合条件的图形 (要求为: 通过饱和度阈值产生的封闭图形, 面积在一定范围内), 先记录其重心横坐标 cur_x 。如果 cur_x 在图像边缘则直行, 否则根据面积和 cur_x 以及颜色计算出目标位置

②如果没检测到图形, 那么根据上次记录的 cur_x , 小于一半视野宽左转, 否则右转

4、PID 控制算法

①使用 PID 控制算法来调整小车的行驶方向, 以使其保持在预定的路径上。

②设置 PID 参数 (比例系数 k_p 、积分系数 k_i 和微分系数 k_d) 来调节算法的响应速度和稳定性。

③计算偏差值, 并根据偏差值调整电机的转速, 使小车朝向目标位置 tgt_x 运动。

5、实验设计与步骤

实验分为两个阶段:

第一阶段: 小车根据图像识别结果和 PID 控制算法调整行驶方向, 保持在预定路径上, 达到避障目的。

第二阶段: 根据运行代码后生成的 `movie.avi` 文件, 诊断故障并调节参数。

可调节参数有:

- ① 电机基础速度和转向控制量 (为了防止抬头, 分为两个阶段系数)
- ② 阶段 1 切换到阶段 2 正常速度所需要的循环数
- ③ 目标位置限制 (越大越晚转)
- ④ 边缘判定阈值 (即离视野边缘多近时直行)
- ⑤ 与正方体目标距离系数 (越大弯越大)

最终达成小车成功完成魔方绕桩挑战

6、实验分析

①小车起步阶段采用初始速度设定两个阶段的方式, 以避免起步时车头翘起的情况。

②图像识别阶段, 通过图像提取出魔方部分, 计算出避障对应的目标位置, 再根据目标位置计算偏差值后, 根据偏差值调整小车的行驶方向。

③PID 控制算法通过比例、积分和微分的组合来实现对小车的行驶方向的精确调节。

④实验中使用的阈值、形态学操作和 PID 参数需要根据实际场景进行调整, 以获得最佳的性能和稳定性。

五、总结与思考:

在魔方避障实验中, 我们小组的整个过程可以分为硬件安装和代码调试两个部分, 在这两部分中我们都有诸多收获:

(一) 硬件安装:

1. 经历了八字寻迹后, 小车的摄像头较高, 刚好与魔方高度平齐, 小车此时的呈像魔方不完整, 我们将摄像头降低后, 此问题得以解决。之后又发现摄像头过低造成了小车呈像画面中有大部分为地毯画面, 不能准确识别障碍物的颜色, 又将小车摄像头适度调高后解决了此问题。
2. 由于我们小车的占空比较高, 所以我们所调的小车参数几乎只有在高电量 (93-100) 时可以正常使用。
3. 小车启动时翘头从而导致小车识别有误, 代码中做了处理后, 就牵涉到小车重心问题, 可在车头加上适当的配重, 解决这一问题。
4. 小车出现原地转圈撞墙, 是由于摄像头没有正对前方, 而是俯视地面所导致, 此时将摄像头抬平即可。

(二) 代码调试 (排除硬件问题后)

1. 小车识别到魔方后, 很早的做出反应或者反应迟钝时, 调节目标坐标限制可以解决这个问题。
2. 小车绕桩时绕大圈, 或者转弯不够, 仍然会碰到魔方时, 可以调节转向限制和与魔方的距离系数来解决。